CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
B. TECH (IT/CE/CSE/AIML)

Chandubhai S Patel Institute of Technology

DEPSTAR

ADVANCED WEB DEVELOPMENT FRAMEWORKS (ITUE301)

Practical 6: Full Stack Integration React + Node + MongoDB

CO/PO	CO1, CO2 / PO3, PO5
Objective	To wire the React frontend to the Node/Express/MongoDB backend into a fully functional full-stack application.
Prerequisites	• Practicals 1–3 completed working React UI with routing and API consumption patterns • Practicals 4–5 completed working Node/Express/MongoDB backend with CRUD
Theory Concepts (Self-Study Reference)	
Coursera Pointer	See Week 6 IBM: Node.js & MongoDB Developing Back-end Database Applications, Module 3 (connecting frontend to backend, end-to-end API calls, full-stack flow).
Architecture / Diagram	

Diagram	`React Frontend (localhost:5173)`
	`  |  fetch/axios calls`
	`  v`
	`Express Backend (localhost:5000)`
	`  |  Mongoose`
	`  v`
	`MongoDB Database`
	
	`Flow: UI Form → POST /tasks → MongoDB → UI re-fetches → updated list rendered`

Problem Definition	
Problem Statement	• Connect the React Task UI to the Node+MongoDB backend built in Practicals 4 and 5. • Replace the GitHub repo data from Practical 3 with task data from your own backend. • The application must support creating, viewing, updating, and deleting tasks from the React interface through API calls. • All data must be persisted in MongoDB confirmed by refreshing the browser and seeing data remain. • Handle loading and error states for every API interaction, not just the initial fetch.
Key Questions / Analysis	• What changes are required on the backend (CORS) to allow the React dev server to call the Express API? • Why should the UI re-fetch or update local state after a successful POST/PUT/DELETE rather than assuming success silently? • What is the risk of not handling errors on write operations (POST/PUT/DELETE) the same way as read operations (GET)?
Supplementary Problems	• Add an optimistic UI update for task creation show the new task in the list immediately, before the server confirms. • Add a confirmation dialog before deleting a task. • Add a toast/notification component that shows success or failure after each operation.
Key Skills Addressed	• Configuring CORS on an Express server for local frontend-backend communication

	• Calling backend CRUD endpoints from React using fetch or axios • Synchronizing UI state with server state after write operations • Handling loading and error states across multiple API interactions in one component
Learning Outcome	Students will be able to integrate a React frontend with a Node/Express/MongoDB backend into a complete, working full-stack application with proper state synchronization.

Lab Session Step-by-Step

Steps

1. Install and configure CORS on the Express backend:

```
npm install cors
const cors = require('cors');
app.use(cors());
```

2. In React, create a central api.js file with the base backend URL:

```
const BASE_URL = 'http://localhost:5000';
export const getTasks = () => fetch(`${BASE_URL}/tasks`).then(res => res.json());
```

3. Replace the Practical 3 GitHub fetch logic with calls to your own /tasks endpoint.
4. Build a task creation form that POSTs to /tasks and updates local state on success.
5. Implement update (PUT) and delete (DELETE) actions, each followed by a state update or re-fetch.
6. Wrap each API call in proper loading/error handling, reusing the pattern from Practical 3.
7. Test the complete flow: create → view → update → delete, refreshing the browser to confirm persistence.

Tools / Technology / Resources Required	• Node.js (v18+), npm • React 18+, Express.js, Mongoose • cors package • Postman (for backend verification), Browser DevTools
Total Hours	• Lab Session: 2 hours • Implementation: 1.5 hours • Testing / Modification: 0.5 hours • Total Engagement: 2 hours

Faculty Teaching Guide

Guidance Notes	• This is the first practical where two dev servers (React on 5173, Express on 5000) must run simultaneously demonstrate running both in separate terminals before students attempt it. • CORS errors are the single most common blocker in this session show what a CORS error looks like in the browser console before students hit it themselves. • Stress that frontend state should be updated based on the backend response, not assumed this is the core integration concept being tested. • Reserve the last 15 minutes for a full end-to-end demo check: create, edit, delete, and refresh.
Common Mistakes	• Forgetting to start the backend server, then debugging the frontend for an unrelated 'fetch failed' error. • Not enabling CORS, resulting in browser console errors blocking all requests. • Updating local UI state optimistically without confirming the backend write actually succeeded. • Hardcoding the backend URL inconsistently across multiple files instead of using one shared base URL constant.

Troubleshooting Guide

Symptom	Likely Cause	Fix

CORS error in browser console	cors middleware not enabled on Express server	Run npm install cors and add app.use(cors()) before route definitions
fetch failed / Network Error	Backend server not running or wrong port	Confirm Express server is running on port 5000 in a separate terminal
UI shows stale data after edit/delete	Local state not updated after successful API call	Re-fetch the task list or manually update state array after each write operation
Data disappears after refresh	Still reading from local component state instead of backend	Confirm GET /tasks is called on mount and populates state from the database, not hardcoded values

Student Assignment / Post Laboratory Work

Tasks	• Review the Theory Concepts section and the assigned Coursera module before next week. • Add a confirmation dialog before deleting a task. • Add a simple toast notification for successful create/update/delete actions.

GitHub Deliverables

Expected Repository Contents	• Two repositories (or one monorepo) clearly separating frontend and backend • Working end-to-end flow: create, read, update, delete tasks from the UI • README with instructions to run both frontend and backend locally • At least one commit specifically for full-stack integration

Rubrics (Practical Evaluation / Viva)

Criteria (Marks)	Description
Conceptual understanding (2 Marks)	Explains what and why, not just what was done
Implementation (4 Marks)	Code written, runs, and is original
Correctness (2 Marks)	Output matches expected result
Reflection (1 marks)	Errors documented and explained
Lab file evidence (1 marks)	Completeness and presentation
Total: 10 marks	**Passing Threshold: 5 / 10 (50%)**